Football RAG Question-Answering System

NLP with Deep Learning

Assignment 3 (Mini-Project 1)

Group Member 1: Suffiyan Asghar Ali (29182)

Group Member 2: Hamza Arif (29234)

Code available at: https://github.com/suffiyan-asghar/nlp_rag.git

Interface:https://huggingface.co/spaces/suffiyan-asghar10/rag_football

Contents

1 Platform Details **2**

2 Data Details **2**
2.1 Domain and Source . . . 2
2.2 Topic Coverage . . . 2
2.3 Corpus Statistics . . . 2

3 Algorithms, Models, and Retrieval Methods **3**
3.1 Chunking Strategies . . . 3
3.1.1 Fixed Chunking . . . 3
3.1.2 Recursive Chunking . . . 3
3.1.3 Semantic Chunking . . . 3
3.2 Embedding Model . . . 3
3.3 Vector Store . . . 4
3.4 Retrieval Pipeline . . . 4
3.4.1 BM25 Sparse Retrieval . . . 4
3.4.2 Dense Retrieval . . . 4
3.4.3 Reciprocal Rank Fusion (RRF) . . . 4
3.4.4 CrossEncoder Re-ranking . . . 4
3.4.5 MMR Deduplication (NEW) . . . 5
3.5 Answer Generation . . . 5
3.6 LLM-as-Judge Prompts . . . 5
3.6.1 Faithfulness: Claim Extraction . . . 5
3.6.2 Faithfulness: Claim Verification . . . 5
3.6.3 Response Relevancy: Alternate Question Generation . . . 6

4 Performance Metrics **6**
4.1 Retrieval Metrics . . . 6
4.2 LLM-as-Judge: Faithfulness and Response Relevancy . . . 7
4.3 Example Judge Outputs . . . 7
4.4 Ablation Study . . . 8
4.5 Latency and Computational Efficiency . . . 9

5 Best Model Selection **9**

6 Reproducibility **10**
6.1 Environment and Dependencies . . . 10
6.2 Configuration Parameters . . . 10
6.3 Step-by-Step Reproduction . . . 11
6.4 Evaluation Reproducibility . . . 11

References **11**

1 Platform Details

All pipeline stages were executed across two platforms, depending on the computational demands of the task.

Data collection, chunking, embedding, and Pinecone upsert were performed on **Kaggle** (free tier) using an NVIDIA Tesla T4 GPU (16 GB VRAM). This environment was chosen because embedding 4,000–5,000 chunks with `BAAI/bge-small-en-v1.5` and cross-encoding with `BAAI/bge-reranker-base` benefit significantly from GPU acceleration. Kaggle secrets were used to securely pass the `PINECONE_API_KEY` and `GROQ_API_KEY` environment variables.

Retrieval experiments, LLM-as-Judge evaluation, and ablation study were also executed on Kaggle due to the need for the same GPU for the CrossEncoder reranker during retrieval.

The deployed web application runs on **Hugging Face Spaces** (free tier, CPU), using Gradio 6. At query time, embedding is performed on CPU using the same lightweight `BAAI/bge-small-en-v1.5` model, while generation and judge LLM calls are routed to the **Groq API** (free tier), making inference latency acceptable without a GPU.

2 Data Details

2.1 Domain and Source

The system targets the **football (soccer) domain**. All documents were fetched from **English Wikipedia** using the `wikipedia` Python library. Wikipedia was chosen because it provides structured, factual, and publicly accessible prose that is well-suited for RAG evaluation.

2.2 Topic Coverage

A total of **≈80 Wikipedia topics** were fetched, spanning:

- **Tournaments and competitions:** FIFA World Cup (multiple editions), UEFA Champions League, UEFA Europa League, FA Cup, EFL Cup, Copa América, Copa Libertadores, Bundesliga, La Liga, Premier League, Serie A, Ligue 1, and others.
- **Clubs:** Real Madrid, FC Barcelona, Manchester City, Liverpool, Chelsea, Bayern Munich, Borussia Dortmund, Juventus, AC Milan, Inter Milan, PSG, AFC Ajax, FC Porto, Atlético Madrid, and others.
- **Players:** Lionel Messi, Cristiano Ronaldo, Kylian Mbappé, Erling Haaland, Neymar, Luka Modrić, Kevin De Bruyne, Karim Benzema, Mohamed Salah, Jude Bellingham, Virgil van Dijk, Robert Lewandowski, Pelé, Diego Maradona, Johan Cruyff, Zinedine Zidane, Ronaldinho, Ronaldo (Brazilian), Harry Kane, Vinicius Jr.
- **National teams:** Argentina, Brazil, France, Germany, Spain, England, Portugal, Italy, Netherlands, Croatia.

2.3 Corpus Statistics

After cleaning and chunking, the corpus statistics are summarised in Table 1.

Table 1: Corpus statistics by chunking strategy.

Strategy	Chunks	Chunk Size	Overlap	Min Chars	Pinecone Namespace
Fixed	4,872	1,000	150	80	`fixed`
Recursive	4,860	1,000	100	80	`recursive`
Semantic	4,998	1,000 max	—	250	`semantic`

Text preprocessing removed Wikipedia section headers (`==...==`), citation brackets (`[1]`), and excess whitespace. Unicode normalisation via NFKD was applied to generate Pinecone-safe ASCII chunk IDs.

3 Algorithms, Models, and Retrieval Methods

3.1 Chunking Strategies

Three distinct chunking strategies were implemented and compared:

3.1.1 Fixed Chunking

Documents were split into non-overlapping windows of 1,000 characters with a 150-character overlap. The `RecursiveCharacterTextSplitter` from LangChain was used with default separators. This strategy is simple and predictable, ensuring uniform chunk sizes at the cost of potentially splitting mid-sentence or mid-thought.

3.1.2 Recursive Chunking

Identical to fixed chunking but with a reduced 100-character overlap and explicit separators prioritised in the order: `\n\n`, `\n`, `". "`, `" "`, `""`. This encourages splits at natural language boundaries (paragraph breaks, then sentence breaks) while still respecting the 1,000-character target size.

3.1.3 Semantic Chunking

Sentences were grouped greedily based on cosine similarity between adjacent sentence embeddings (using `all-MiniLM-L6-v2`). A similarity threshold of 0.32 was used as the break point: if consecutive sentences fell below this threshold, a new chunk was started. A hard ceiling of 1,000 characters and a floor of 250 characters were enforced. Chunks exceeding the ceiling were recursively split using the fixed strategy.

3.2 Embedding Model

Query and document embeddings were generated using `BAAI/bge-small-en-v1.5` (384 dimensions, cosine similarity metric). This model was selected because it offers strong retrieval performance for its small size (≈33M parameters), making it suitable for both the Kaggle upsert phase (GPU) and the Hugging Face Spaces query phase (CPU). All embeddings were L2-normalised before upsert and query.

Following the BGE retrieval best practice, a **query instruction prefix** was prepended at query time:

```
"Represent this sentence for searching relevant passages: {query}"
```

Document embeddings were encoded without this prefix, as is standard for BGE models.

3.3 Vector Store

Pinecone (Serverless, free tier, AWS us-east-1) was used as the cloud vector database. A single index named `sports-rag-v2` (dimension 384, cosine metric) was created, with each chunking strategy stored in its own **namespace** (`fixed`, `recursive`, `semantic`). Chunk text and metadata (source, URL, fetched date) were stored in Pinecone vector metadata to avoid needing a separate document store.

3.4 Retrieval Pipeline

The full retrieval pipeline consists of four stages, illustrated conceptually below:

$$\textbf{Query} \rightarrow \textbf{Hybrid (BM25 + Dense} \xrightarrow{\textbf{RRF}} \textbf{Fused)} \rightarrow \textbf{CrossEncoder Rerank} \rightarrow \textbf{MMR Dedup} \rightarrow \textbf{Top-5 Context}$$

3.4.1 BM25 Sparse Retrieval

A `BM25Okapi` index was built in-memory from each chunking strategy's corpus using simple alphanumeric tokenisation. For each query, the top-30 candidates were retrieved. BM25 provides strong lexical matching and is particularly effective for queries with precise named entities (e.g. player names, club names, competition names).

3.4.2 Dense Retrieval

The query (with BGE prefix) was encoded and queried against Pinecone with `top_k=30`. Dense retrieval captures semantic similarity and is robust to paraphrasing and vocabulary mismatch.

3.4.3 Reciprocal Rank Fusion (RRF)

Both result lists were merged using RRF with $k = 60$:

$$\text{RRF_score}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}$$

where R is the set of retrieval systems and $\text{rank}_r(d)$ is the rank of document d in list r. This fusion strategy is parameter-free, robust to score scale differences, and consistently outperforms individual retrievers.

3.4.4 CrossEncoder Re-ranking

The top-30 fused candidates were re-ranked using `BAAI/bge-reranker-base`, a cross-encoder that jointly encodes the query–document pair to produce a relevance score. This model provides higher-quality relevance judgements than the bi-encoder at the cost of increased latency (it cannot be pre-computed).

3.4.5 MMR Deduplication (NEW)

After re-ranking, **Maximal Marginal Relevance (MMR)** was applied to select the final top-5 context chunks. MMR iteratively selects the candidate that maximises:

$$\mathrm{MMR}(d) = \lambda \cdot \mathrm{sim}(d, q) - (1 - \lambda) \cdot \max_{s \in S} \mathrm{sim}(d, s)$$

where $\lambda = 0.6$ balances relevance against diversity, q is the query embedding, and S is the set of already-selected documents. This prevents multiple near-duplicate chunks from the same Wikipedia article from dominating the context window.

3.5 Answer Generation

Answers were generated using **Groq's** `llama-3.1-8b-instant` via the OpenAI-compatible Groq API. Groq's inference infrastructure delivers sub-second token generation, making it practical for real-time use in the web interface. The generation prompt was:

```
Answer the question using ONLY the context provided below.
Write a complete, fluent sentence.
If the context does not contain enough information,
respond with exactly: I don't know.

Question: {question}

Context:
{context}

Answer:
```

The strict "use ONLY the context" instruction grounds the answer in retrieved evidence and enables faithful claim verification. Unanswerable queries (those involving events after the corpus snapshot or information not in the corpus) are expected to trigger the "I don't know" abstention.

3.6 LLM-as-Judge Prompts

3.6.1 Faithfulness: Claim Extraction

```
[System] You extract factual atomic claims from answers.

[User] Extract atomic factual claims from the answer. Return JSON
with key 'claims' as a list of short strings. Return an empty
list if the answer contains no factual content.

Answer:
{answer}
```

3.6.2 Faithfulness: Claim Verification

```
[System] You verify whether a claim is supported by the provided
context only.
```

```
[User] Verify if the claim is supported by the context only.
Return JSON with keys: label ('supported' or 'unsupported'),
rationale (1-2 lines), evidence (short phrase or empty).

Question: {question}
Claim: {claim}
Context: {context}
```

3.6.3 Response Relevancy: Alternate Question Generation

```
[System] You generate alternative user queries for the given
answer could respond to.

[User] Generate exactly 3 different questions this answer could
respond to. Return JSON with key 'questions' as a list of
exactly 3 strings.

Answer:
{answer}
```

All judge calls used `temperature=0` and `response_format={''type'': ''json_object''}` to ensure structured, reproducible outputs. Rate-limit errors were handled with exponential back-off (up to 6 retries).

4 Performance Metrics

4.1 Retrieval Metrics

Six retrieval configurations were evaluated on a fixed set of 60 answerable queries with known relevant chunk IDs. Table 2 reports Recall@5, Recall@10, MRR@10, and average retrieval latency.

Table 2: Retrieval experiment results across all six configurations.

Experiment	Recall@5	Recall@10	MRR@10	Avg Ret. Time (s)
rowgray hybrid_fixed	**0.833**	**0.967**	**0.706**	0.069
hybrid_recursive	0.833	0.933	0.703	0.066
hybrid_semantic	0.817	0.917	0.676	0.067
bm25_semantic	0.600	0.717	0.417	0.012
bm25_fixed	0.533	0.717	0.392	0.011
bm25_recursive	0.567	0.700	0.390	0.011

The hybrid configurations uniformly outperform BM25-only retrievers across all metrics. **hybrid_fixed** achieves the highest Recall@5 (0.833) and MRR@10 (0.706), while maintaining a retrieval latency of only 69 ms. The sparse-only BM25 approaches suffer substantially lower MRR (≈0.39–0.42), indicating that keyword matching alone misses relevant passages for many football queries that require semantic understanding.

4.2 LLM-as-Judge: Faithfulness and Response Relevancy

The LLM-as-Judge evaluation was performed on the best retriever (`hybrid_fixed`) using 10 non-abstention queries. Table 3 summarises the per-query scores.

Table 3: LLM-as-Judge results for `hybrid_fixed` retriever (10 queries).

Question	**Claims**	**Supported**	**Faith.**	**Relev.**
What is Ballon d'Or known for…?	3	3	1.000	0.618
Give a brief overview of Atlético Madrid…	4	4	1.000	0.753
How many goals did Ronaldo (Brazilian)…?	2	1	0.500	0.888
England national football team represents…	2	2	1.000	0.943
What is Chelsea FC known for…?	3	3	1.000	0.833
How many UCL titles has Real Madrid won?	2	2	1.000	0.890
What type of tournament is FA Cup?	1	1	1.000	0.892
Which club has more CL titles—AC Milan…?	1	0	0.000	0.953
Give a brief overview of FIFA…?	3	3	1.000	0.768
Give a brief football profile of FC Barcelona	6	6	1.000	0.742
Average	2.7	2.5	**0.850**	**0.828**

The system achieves an average faithfulness of **0.850** and average relevancy of **0.828**. The one faithfulness failure (AC Milan vs. Liverpool) occurred because the retrieved context confirmed AC Milan has 7 Champions League titles, but the generated answer made a comparative claim ("more than Liverpool") which the judge could not verify directly from the context. This is a known limitation of comparative queries. The one partial failure (Ronaldo's 15 goals) occurred because the judge correctly identified that stating "played 19 World Cup matches" without explicit context support is an unverifiable sub-claim.

4.3 Example Judge Outputs

The following three examples, reported in Table 4, illustrate the claim extraction and verification process in detail.

Table 4: Three example LLM-as-Judge evaluations showing claim extraction and verification.

Question	**Answer**	**Claims & Verdict**	**Scores**
What is the Ballon d'Or known for in football?	The Ballon d'Or is known for honouring the player deemed to have performed the best over the previous season in football.	"The Ballon d'Or honours the best football player." "The Ballon d'Or is for football players." "Awarded to the best player over the previous season."	Faith: 1.00 Relev: 0.618
Give a brief overview of Atlético Madrid in football.	Atlético Madrid is one of the most successful Spanish clubs, having won 11 La Liga titles and the UEFA Europa League in 2010, 2012, and 2018.	"Won 11 La Liga titles" "Won UEFA Europa League in 2010" "Won UEFA Europa League in 2012" "Won UEFA Europa League in 2018"	Faith: 1.00 Relev: 0.753
How many goals did Ronaldo (the Brazilian) score across all World Cup tournaments?	Ronaldo finished with fifteen goals in nineteen World Cup matches.	"Ronaldo finished with fifteen goals." × "Ronaldo played nineteen World Cup matches." (claim in context, but the judge reads it as a distinct claim without a direct statement)	Faith: 0.50 Relev: 0.888

4.4 Ablation Study

Table 5 presents the full ablation across all six combinations of chunking strategy and retrieval variant, evaluated on 8 queries using the LLM-as-Judge protocol.

Table 5: Ablation study results: 3 chunking strategies × 2 retrieval variants (8 queries each).

Chunking	Retrieval	Faith.	Relev.	Avg Ret. (s)	Avg Inf. (s)	Avg Total (s)	QPS
rowgray recursive	semantic_only	**0.938**	**0.851**	0.062	6.112	6.174	0.162
fixed	semantic_only	0.938	0.840	0.099	6.222	6.321	0.158
semantic	semantic_only	0.875	0.818	0.053	5.601	5.654	0.177
fixed	hybrid+rerank	0.813	0.846	0.814	5.384	6.199	0.161
recursive	hybrid+rerank	0.813	0.840	0.796	4.704	5.500	0.182
semantic	hybrid+rerank	0.708	0.846	0.809	5.363	6.172	0.162

Key observations from Table 5:

- **Chunking:** Recursive and fixed chunking consistently outperform semantic chunking on faithfulness. Semantic chunking's variable chunk sizes introduce noise—some chunks are too short to provide sufficient context, while others exceed the model's effective context window.
- **Retrieval:** Semantic-only (dense-only) retrieval achieves *higher* faithfulness than hybrid+rerank in this ablation. This is counter-intuitive and is explained by the small ablation size ($n = 8$ queries): on this question set, dense retrieval already recovers the most relevant passages, and the BM25 leg of hybrid retrieval introduces some irrelevant candidates that survive into the final context.
- **Latency:** Semantic-only retrieval is dramatically faster (avg 0.05–0.10 s) versus hybrid+rerank (avg 0.80 s), because it skips BM25 scoring, RRF fusion, and CrossEncoder inference. The inference (generation) time dominates overall latency in both cases at ≈5–6 s, reflecting Groq API response time.
- **Relevancy:** Hybrid+rerank achieves comparable or slightly higher relevancy scores than semantic-only, suggesting that the diverse retrieval sources improve response scope even if some faithfulness is sacrificed.

4.5 Latency and Computational Efficiency

Table 6 summarises the end-to-end system latency as measured across the ablation study.

Table 6: Latency breakdown across retrieval variants.

Variant	Avg Ret. (s)	P95 Ret. (s)	Avg Inf. (s)	Avg Total (s)	QPS
semantic_only	0.071	0.203	5.978	6.049	0.163
hybrid+rerank	0.806	0.871	5.150	5.957	0.168

Retrieval accounts for only 1–13% of total latency. The dominant cost is LLM generation, which takes ≈5–6 s due to network round-trips to the Groq API. On a locally hosted LLM, this could be reduced to < 1 s.

5 Best Model Selection

Based on the retrieval experiments (Table 2), LLM-as-Judge evaluation (Table 3), and ablation study (Table 5), the best overall configuration is:

Component	Selection
Chunking strategy	**Recursive** (CHUNK_SIZE=1,000, OVERLAP=100)
Embedding model	**BAAI/bge-small-en-v1.5** with BGE query prefix
Retrieval	**Hybrid (BM25 + Dense, RRF)** for offline evaluation
Re-ranker	**BAAI/bge-reranker-base** (CrossEncoder)
Post-processing	**MMR** ($\lambda = 0.6$, top-5)
Generation LLM	**Groq llama-3.1-8b-instant**

Justification:

1. **Retrieval metrics** (Table 2) show that `hybrid_fixed` and `hybrid_recursive` are tied at Recall@5=0.833 and nearly tied on MRR@10 (0.706 vs. 0.703). We prefer **recursive chunking** because it yields slightly better retrieval time (66 ms vs. 69 ms) and its sentence-boundary-respecting splits produce more coherent chunks.
2. **Ablation study** (Table 5) shows that `recursive / semantic_only` achieves the highest faithfulness (0.938) and highest relevancy (0.851) in the ablation. However, the assignment requires Hybrid+Re-ranking, and `recursive / hybrid+rerank` is the best hybrid configuration (faithfulness 0.813, relevancy 0.840, total time 5.50 s).
3. **MMR deduplication** is justified empirically: without MMR, the top-5 context chunks frequently contained 3–4 chunks from the same Wikipedia article, saturating the context with redundant information. With $\lambda = 0.6$, diverse source articles are represented while maintaining high relevance.
4. **Groq llama-3.1-8b-instant** was chosen over alternatives (Mistral-7B, TinyLlama) because Groq's hardware delivers tokens at >300 tok/s, keeping generation latency under 6 s even on the free tier, while the 8B model size provides sufficient reasoning capability for factual football QA.

6 Reproducibility

6.1 Environment and Dependencies

```
# Core dependencies
pip install wikipedia langchain langchain-text-splitters \
            sentence-transformers pinecone rank-bm25 \
            openai gradio>=6.0.0 numpy pandas tqdm \
            transformers accelerate torch
```

Python version: 3.12. All models are downloaded automatically from HuggingFace Hub on the first run.

6.2 Configuration Parameters

All hyperparameters are centralised in the notebook's **Global Config** cell (Cell 4). The key values are:

```
INDEX_NAME       = "sports-rag-v2"   # Pinecone index
EMBEDDING_MODEL  = "BAAI/bge-small-en-v1.5"  # dim=384
CROSS_ENCODER    = "BAAI/bge-reranker-base"
JUDGE_MODEL      = "llama-3.1-8b-instant"     # via Groq
CHUNK_SIZE       = 1000
```

```
CHUNK_OVERLAP_FIXED = 150
CHUNK_OVERLAP_REC   = 100
TOP_K               = 10
RRF_K               = 60
DENSE_TOP_K         = 30
RERANK_TOP_K        = 30
QA_CONTEXT_TOP_K = 5
MMR_LAMBDA          = 0.6
```

6.3 Step-by-Step Reproduction

1. Create a Pinecone account and obtain an API key. Create a Groq account and obtain a free API key.
2. On Kaggle, add `PINECONE_API_KEY` and `GROQ_API_KEY` as secrets.
3. Run the notebook `football-rag-complete.ipynb` end-to-end. Sections 3–6 fetch Wikipedia articles, chunk them, embed them, and upsert to Pinecone. This takes ≈45–60 minutes on a T4 GPU.
4. Section 7 runs a Pinecone sanity check to confirm all three namespaces are populated.
5. Sections 8–12 build BM25 indexes, run retrieval experiments, and select winners.
6. Sections 13–14 run QA generation and LLM-as-Judge evaluation.
7. Section 15 runs the ablation study.
8. Section 16 saves all outputs as CSV files to `/kaggle/working/outputs/`.
9. For deployment, place `app.py`, `retriever.py`, and `requirements.txt` in a GitHub repository and connect to a Hugging Face Space. Set `PINECONE_API_KEY` and `GROQ_API_KEY` as HF Space secrets.

6.4 Evaluation Reproducibility

All judge evaluations used `temperature=0`, ensuring deterministic outputs for a given model version. The evaluation question bank (`evaluation_question_bank.csv`) is included in the submission, containing 100 questions with known relevance labels, question types (`person`, `club`, `competition`, `country-team`, `general`, `unanswerable`), and expected abstention flags.

References

[1] Wikipedia Python library. https://pypi.org/project/wikipedia/

[2] LangChain Text Splitters. https://python.langchain.com/docs/modules/data_connection/document_transformers/

[3] Pinecone Documentation. https://docs.pinecone.io/

[4] BAAI/bge-small-en-v1.5. https://huggingface.co/BAAI/bge-small-en-v1.5

[5] BAAI/bge-reranker-base. https://huggingface.co/BAAI/bge-reranker-base

[6] Cormack, G.V., Clarke, C.L.A., and Buettcher, S. (2009). Reciprocal rank fusion outperforms condorcet and individual rank learning methods. *SIGIR '09*.

[7] Carbonell, J. and Goldstein, J. (1998). The use of MMR, diversity-based reranking for reordering documents and producing summaries. *SIGIR '98*.

[8] Meta AI (2024). Llama 3. https://ai.meta.com/blog/meta-llama-3/

[9] Groq API Documentation. https://console.groq.com/docs/

[10] Gradio Documentation. https://www.gradio.app/docs/
[11] Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023). RAGAS: Automated Evaluation of Retrieval Augmented Generation. *arXiv:2309.15217*.
[12] Xiao, S. et al. (2023). C-Pack: Packaged Resources To Advance General Chinese Embedding. *arXiv:2309.07597*. (BGE model family paper)